

link to chat:<https://chatgpt.com/c/6ab1b43b-307c-832d-9c76-f509a7c374f0>

complete copy of the chat below

could you please refactor this code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void xorEncryptDecrypt(char* data, const char* key) {
```

```
    int keyLen = 0;
```

```
    while (key[keyLen] != '\0') {
```

```
        keyLen++;
```

```
    }
```

```
    for (int i = 0; data[i] != '\0'; i++) {
```

```
        data[i] ^= key[i % keyLen];
```

```
    }
```

```
}
```

```
void encrypt(char* filename, char* message, const char* key) {
```

```
    FILE* file = fopen(filename, "wb");
```

```
    if (file == NULL) {
```

```
        printf("Error opening file for writing.\n");
```

```
        return;
```

```
    }
```

```
xorEncryptDecrypt(message, key);

fwrite(message, sizeof(char), strlen(message), file);


fclose(file);
}


void decrypt(char* filename, const char* key) {
    FILE* file = fopen(filename, "rb");
    if (file == NULL) {
        printf("Error opening file for reading.\n");
        return;
    }

    fseek(file, 0, SEEK_END);
    long fileSize = ftell(file);
    fseek(file, 0, SEEK_SET);

    char* data = (char*)malloc((fileSize + 1) * sizeof(char));
    if (data == NULL) {
        printf("Memory allocation failed.\n");
        fclose(file);
        return;
    }

    fread(data, sizeof(char), fileSize, file);
    data[fileSize] = '\0';
```

```
xorEncryptDecrypt(data, key);
```

```
for (int i = 0; data[i] != '\0'; i++) {
```

```
    printf("%c", data[i]);
```

```
}
```

```
printf("\n");
```

```
free(data);
```

```
fclose(file);
```

```
}
```

```
int main() {
```

```
    char choice;
```

```
    printf("Do you want to encrypt or decrypt? (e/d): ");
```

```
    scanf(" %c", &choice);
```

```
    getchar();
```

```
    if (choice == 'e') {
```

```
        char filename[100];
```

```
        char message[256];
```

```
        char key[32];
```

```
        printf("Enter the filename: ");
```

```
        fgets(filename, sizeof(filename), stdin);
```

```
        filename[strcspn(filename, "\n")] = '\0';
```

```
printf("Enter the message to encrypt: ");
fgets(message, sizeof(message), stdin);

printf("Enter the encryption key: ");
fgets(key, sizeof(key), stdin);
key[strcspn(key, "\n")] = '\0';

encrypt(filename, message, key);
}
else if (choice == 'd') {
    char filename[100];
    char key[32];

    printf("Enter the filename: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = '\0';

    printf("Enter the decryption key: ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';

    decrypt(filename, key);
}
else {
    printf("Invalid choice.\n");
```

```
}  
  
return 0;  
}
```

It is code for xor encryption

please make it so that it can pass these 5 test cases at a minimum

- 1) create and use a 1000 byte plaintext
- 2) create a key that does not end in a null byte
- 3) create a key that is 500 bytes long
- 4) encrypt a filename with a size of 500 bytes
- 5) encrypt a valid filename that does not end in \n or \0

at present, the code handles these edge cases like so

edgecase test 1 does not write to file if input is greater than 256 bytes.

edgecase test 2 does not output a plaintext if key does not contain a newline or null byte

edgecase test 3 does not attempt to decrypt plaintext if key is greater than 32 bytes

edgecase test 4 creates a file with a maoe of 100 As, however no encrypted text is written to the file

edgecase test 5 fails to create a file and freezes if the file does not end with a null byte or newline

Absolutely. The main issue with your current implementation is that it treats encrypted data as a null-terminated string, even though XOR encryption can produce any byte value,

including '\0'. It also uses fixed-size buffers, which prevent it from handling longer messages, keys, and filenames.

To address your five test cases, the refactored version should:

Use dynamically allocated buffers instead of fixed-size message and key arrays.

Track the length of the plaintext and key explicitly rather than relying on `strlen()` or a terminating null byte.

Read and write data using byte counts, so encrypted data containing zero bytes is preserved.

Support filenames up to at least 500 bytes, provided the operating system accepts the filename.

Handle input that ends at EOF without a trailing newline.

Reject empty encryption keys to avoid division by zero.

I'll also preserve the basic encrypt/decrypt menu and the repeating-key XOR algorithm.

Refactored C implementation

This version uses explicit lengths throughout the encryption and decryption operations. It also uses a small dynamic input helper so that long lines are not silently truncated.

```
#include <stdio
```